FIAP / 2TDSPF / 2026

ClyvoVet
Banco de dados
Sprint 4

Mastering Relational and Non-Relational Database

Continuação da Sprint 3 • Nota anterior: 100

Integrantes

Nome	RM
Carlos Alberto Guedes Neto	566022
Eduardo Novaes Mollo	561515
Luan Peixoto Marins Rocha	562258
Mathaus Victor Souza Marcelino	564146
Vinicius Luis Exposito Morassi	563340

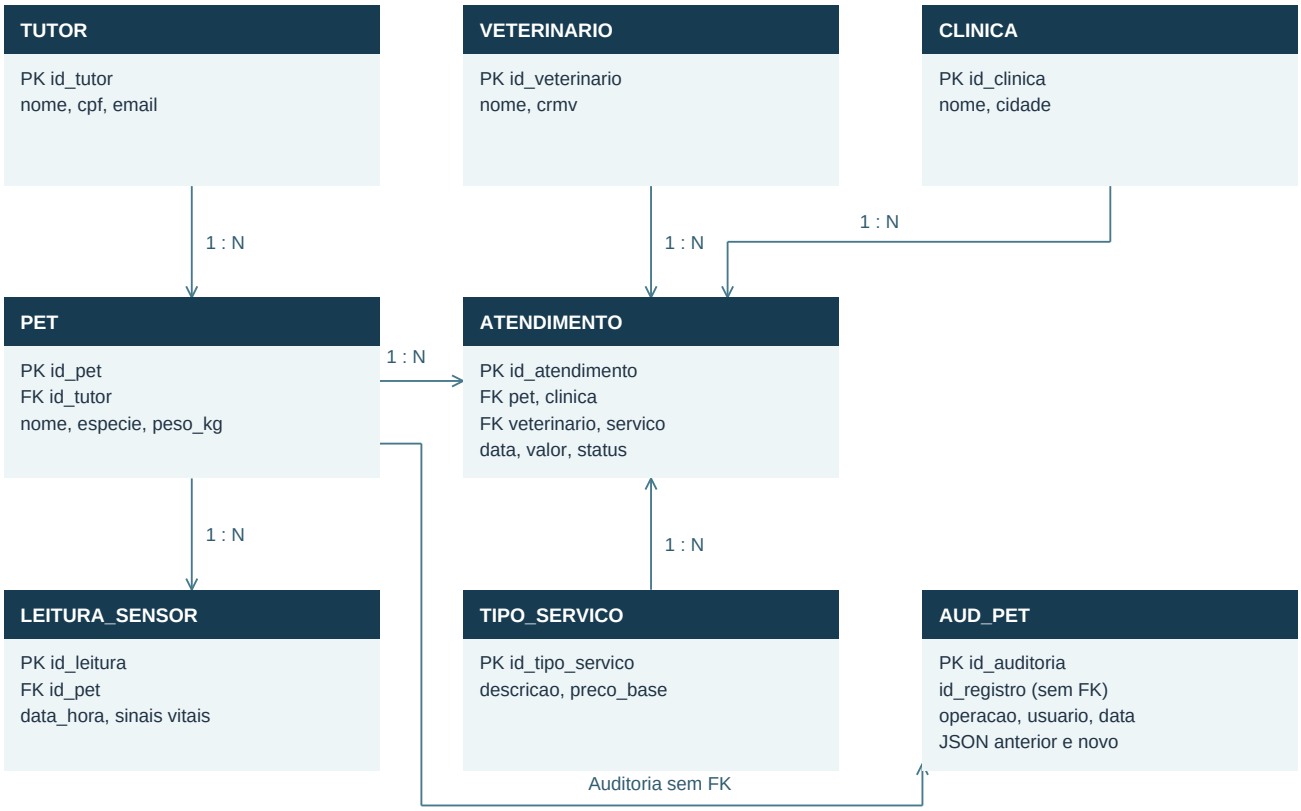
Modelagem relacional • Packages PL/SQL • Integração Java
Exportação JSON • Estrutura e importação MongoDB

Entrega: 04 de novembro de 2026

A documentação descreve o código entregue. Evidências externas só são consideradas concluídas depois da execução real no Oracle FIAP e no MongoDB.

Modelo lógico relacional

Continuidade do modelo aprovado com nota 100. O núcleo clínico conserva tutores, pets, clínicas, veterinários, serviços, atendimentos, sensores e auditoria. A evolução incorpora autenticação e acompanhamento clínico já utilizados pelo backend Java.



O diagrama apresenta o núcleo do domínio. As relações adicionais de cuidado e acesso estão detalhadas na página seguinte. A auditoria usa uma referência histórica, não uma chave estrangeira.

Relacionamentos e integridade

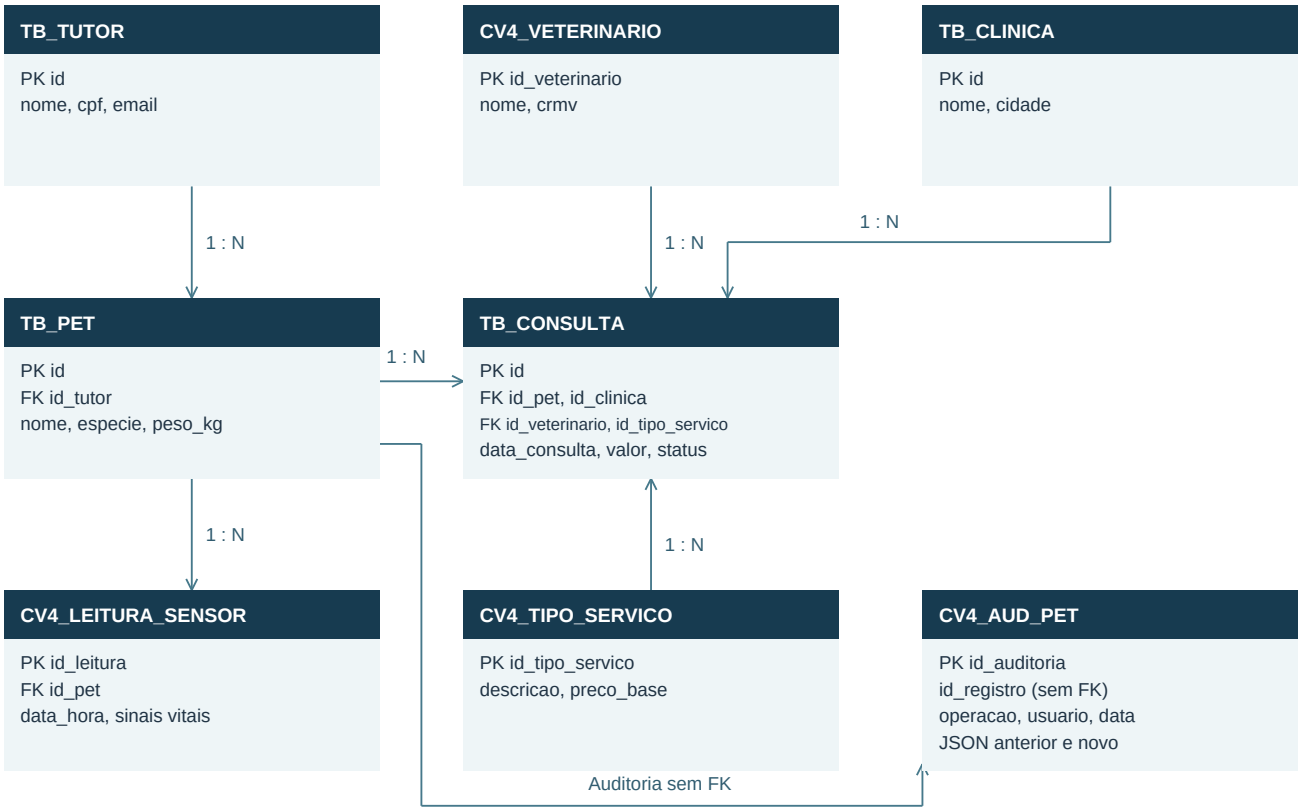
Relacionamento	Cardinalidade	Regra
Tutor > Pet	1 : N	Cada pet tem um tutor obrigatório.
Clínica > Pet principal	0..1 : N	Clínica principal é opcional no pet.
Pet > Atendimento	1 : N	Cada atendimento possui um pet.
Clínica > Atendimento	1 : N	Cada atendimento ocorre em uma clínica.
Veterinário > Atendimento	1 : N	FK obrigatória; catálogo validado pelo package.
Tipo de serviço > Atendimento	1 : N	FK obrigatória; classificação compatível com o Java.
Pet > Leitura de sensor	1 : N	As leituras alimentam o cálculo aprovado da Sprint 3.
Pet > Auditoria	Referência histórica	Sem FK: a trilha deve sobreviver ao DELETE do pet.
Pet > Evento de saúde	1 : N	Consulta de origem opcional.
Pet > Vacina	1 : N	Clínica da aplicação opcional.
Pet > Medicamento	1 : N	Consulta de origem opcional.
Tutor > Usuário	0..1 : N	Usuário pode ser vinculado a um tutor.
Usuário / Perfil	N : N	Associação TB_USUARIO_PERFIL com PK composta.

Dependências e compatibilidade

Veterinário e serviço são entidades próprias, com FKs reais no atendimento. Os campos textuais veterinario e tipo_consulta continuam presentes por compatibilidade com o Java existente. O trigger valida o catálogo e impede uma associação contraditória. Essa redundância controlada está explicitada no modelo físico.

Modelo físico Oracle

Banco Oracle 19c ou superior. O backend JPA utiliza TB_*; os complementos CV4_* preservam os objetos da Sprint 3 existentes no mesmo schema FIAP. As views não armazenam cópias dos dados.



Entidade anterior	Objeto atual	Origem compartilhada
CLINICA	CV4_CLINICA (view)	TB_CLINICA
TUTOR	CV4_TUTOR (view)	TB_TUTOR
PET	CV4_PET (view)	TB_PET
ATENDIMENTO	CV4_ATENDIMENTO (view)	TB_CONSULTA
VETERINARIO	CV4_VETERINARIO	Tabela
TIPO_SERVICO	CV4_TIPO_SERVICO	Tabela
LEITURA_SENSOR	CV4_LEITURA_SENSOR	Tabela / FK TB_PET
AUD_PET	CV4_AUD_PET	Tabela / histórico

Dicionário físico / 01

TB_TUTOR

Coluna	Tipo Oracle	Regras
id	NUMBER(19)	IDENTITY / PK
nome	VARCHAR2(100)	NOT NULL
cpf	VARCHAR2(11)	NOT NULL
email	VARCHAR2(150)	NOT NULL
telefone	VARCHAR2(11)	NOT NULL
data_nascimento	DATE	NOT NULL
ativo	NUMBER(1)	NOT NULL

TB_CLINICA

Coluna	Tipo Oracle	Regras
id	NUMBER(19)	IDENTITY / PK
nome	VARCHAR2(150)	NOT NULL
cnpj	VARCHAR2(14)	NOT NULL
telefone	VARCHAR2(11)	NOT NULL
email	VARCHAR2(150)	NOT NULL
endereco	VARCHAR2(300)	NOT NULL
cidade	VARCHAR2(100)	NOT NULL
estado	VARCHAR2(2)	NOT NULL
ativo	NUMBER(1)	NOT NULL

Dicionário físico / 02

TB_PET

Coluna	Tipo Oracle	Regras
id	NUMBER(19)	IDENTITY / PK
nome	VARCHAR2(80)	NOT NULL
especie	VARCHAR2(20)	NOT NULL
raca	VARCHAR2(80)	NOT NULL
data_nascimento	DATE	NOT NULL
peso_kg	NUMBER(6,2)	Opcional / ver constraints
sexo	VARCHAR2(10)	Opcional / ver constraints
castrado	NUMBER(1)	Opcional / ver constraints
microchip	VARCHAR2(20)	Opcional / ver constraints
observacoes	VARCHAR2(500)	Opcional / ver constraints
ativo	NUMBER(1)	NOT NULL
id_tutor	NUMBER(19)	NOT NULL
id_clinica_principal	NUMBER(19)	Opcional / ver constraints

TB_CONSULTA

Coluna	Tipo Oracle	Regras
id	NUMBER(19)	IDENTITY / PK
data_consulta	DATE	NOT NULL
status	VARCHAR2(20)	NOT NULL
tipo_consulta	VARCHAR2(30)	NOT NULL
veterinario	VARCHAR2(100)	NOT NULL
diagnostico	VARCHAR2(1000)	Opcional / ver constraints
prescricao	VARCHAR2(1000)	Opcional / ver constraints
observacoes	VARCHAR2(500)	Opcional / ver constraints
valor	NUMBER(10,2)	Opcional / ver constraints
data_retorno	DATE	Opcional / ver constraints
criado_em	TIMESTAMP	NOT NULL

Coluna	Tipo Oracle	Regras
id_pet	NUMBER(19)	NOT NULL
id_clinica	NUMBER(19)	NOT NULL
motivo_cancelamento	VARCHAR2(300)	Opcional
id_veterinario	NUMBER(6)	FK / NOT NULL
id_tipo_servico	NUMBER(6)	FK / NOT NULL

Dicionário físico / 03

CV4_VETERINARIO

Coluna	Tipo Oracle	Regras
id_veterinario	NUMBER(6)	PK
nome	VARCHAR2(100)	NOT NULL
crmv	VARCHAR2(20)	NOT NULL
especialidade	VARCHAR2(60)	Opcional / ver constraints

CV4_TIPO_SERVICO

Coluna	Tipo Oracle	Regras
id_tipo_servico	NUMBER(6)	PK
descricao	VARCHAR2(60)	NOT NULL
preco_base	NUMBER(10,2)	NOT NULL
codigo_app	VARCHAR2(30)	NOT NULL

CV4_LEITURA_SENSOR

Coluna	Tipo Oracle	Regras
id_leitura	NUMBER(10)	PK
id_pet	NUMBER(19)	NOT NULL, FK
dispositivo	VARCHAR2(30)	NOT NULL
data_hora	TIMESTAMP	NOT NULL
temperatura	NUMBER(4,1)	NOT NULL
freq_cardiaca	NUMBER(4)	NOT NULL
spo2	NUMBER(5,1)	NOT NULL
atividade	NUMBER(5,1)	NOT NULL

Dicionário físico / 04

CV4_AUD_PET

Coluna	Tipo Oracle	Regras
id_auditoria	NUMBER	IDENTITY / PK, PK
id_registro	NUMBER(19)	Opcional / ver constraints
tipo_operacao	VARCHAR2(10)	NOT NULL
nome_usuario	VARCHAR2(128)	NOT NULL
data_hora	TIMESTAMP	NOT NULL
valor_anterior	CLOB	Opcional / ver constraints
valor_novo	CLOB	Opcional / ver constraints
nome_tabela	VARCHAR2(30)	NOT NULL / default TB_PET
usuario_so	VARCHAR2(100)	Opcional
maquina	VARCHAR2(100)	Opcional
usuario_aplicacao	VARCHAR2(100)	Opcional

TB_EVENTO_SAUDE

Coluna	Tipo Oracle	Regras
id	NUMBER(19)	IDENTITY / PK
tipo_evento	VARCHAR2(30)	NOT NULL
descricao	VARCHAR2(500)	NOT NULL
data_evento	DATE	NOT NULL
data_proxima_acao	DATE	Opcional / ver constraints
concluido	NUMBER(1)	NOT NULL
criado_em	TIMESTAMP	NOT NULL
id_pet	NUMBER(19)	NOT NULL
id_consulta	NUMBER(19)	Opcional / ver constraints

Dicionário físico / 05

TB_VACINA

Coluna	Tipo Oracle	Regras
id	NUMBER(19)	IDENTITY / PK
nome	VARCHAR2(100)	NOT NULL
fabricante	VARCHAR2(100)	NOT NULL
lote	VARCHAR2(50)	Opcional / ver constraints
data_aplicacao	DATE	NOT NULL
data_validade	DATE	NOT NULL
proxima_dose	DATE	Opcional / ver constraints
criado_em	TIMESTAMP	NOT NULL
id_pet	NUMBER(19)	NOT NULL
id_clinica	NUMBER(19)	Opcional / ver constraints

TB_MEDICAMENTO

Coluna	Tipo Oracle	Regras
id	NUMBER(19)	IDENTITY / PK
nome	VARCHAR2(100)	NOT NULL
principio_ativo	VARCHAR2(100)	Opcional / ver constraints
dosagem	VARCHAR2(100)	NOT NULL
frequencia	VARCHAR2(100)	NOT NULL
data_inicio	DATE	NOT NULL
data_fim	DATE	Opcional / ver constraints
uso_continuo	NUMBER(1)	NOT NULL
observacoes	VARCHAR2(500)	Opcional / ver constraints
id_pet	NUMBER(19)	NOT NULL
id_consulta	NUMBER(19)	Opcional / ver constraints

Dicionário físico / 06

TB_USUARIO

Coluna	Tipo Oracle	Regras
id	NUMBER(19)	IDENTITY / PK
username	VARCHAR2(60)	NOT NULL
senha	VARCHAR2(100)	NOT NULL
nome_exibicao	VARCHAR2(100)	NOT NULL
ativo	NUMBER(1)	NOT NULL
data_criacao	DATE	NOT NULL
id_tutor	NUMBER(19)	Opcional / ver constraints

TB_PERFIL

Coluna	Tipo Oracle	Regras
id	NUMBER(19)	IDENTITY / PK
descricao	VARCHAR2(20)	NOT NULL

TB_USUARIO_PERFIL

Coluna	Tipo Oracle	Regras
id_usuario	NUMBER(19)	NOT NULL
id_perfil	NUMBER(19)	NOT NULL

Chaves, índices e regras

Tabela de origem	Tabela referenciada
TB_USUARIO	TB_TUTOR
TB_USUARIO_PERFIL	TB_USUARIO
TB_USUARIO_PERFIL	TB_PERFIL
TB_PET	TB_TUTOR
TB_PET	TB_CLINICA
TB_CONSULTA	TB_PET
TB_CONSULTA	TB_CLINICA
TB_CONSULTA	CV4_VETERINARIO
TB_CONSULTA	CV4_TIPO_SERVICO
CV4_LEITURA_SENSOR	TB_PET
TB_EVENTO_SAUDE	TB_PET
TB_EVENTO_SAUDE	TB_CONSULTA
TB_VACINA	TB_PET
TB_VACINA	TB_CLINICA
TB_MEDICAMENTO	TB_PET
TB_MEDICAMENTO	TB_CONSULTA

PKs: coluna id nas tabelas TB_*, id_veterinario, id_tipo_servico, id_leitura e id_auditoria nos complementos; TB_USUARIO_PERFIL usa (id_usuario,id_perfil).

Unicidade: username, descrição do perfil, CPF e email do tutor, CNPJ da clínica, microchip do pet, CRMV e nome normalizado do veterinário.

Índices: pet/tutor, consulta/pet/data, consulta/status, consulta/veterinário, consulta/serviço, vacina/pet, medicamento/pet, evento/pet, leitura/pet/data e auditoria/registro/data.

CHECKs de sensores: temperatura 20-50, frequência 1-400, SpO2 e atividade 0-100. Auditoria aceita INSERT, UPDATE e DELETE. Essas faixas são validações de entrada da demonstração.

Geração de IDs: IDENTITY nas tabelas do Java e auditoria; CV4_SEQ_LEITURA para leituras. Após a carga com IDs explícitos, START WITH LIMIT VALUE sincroniza as identidades.

Validação técnica executada

Execução real concluída em 01/11/2026 no terminal Windows, com Oracle FIAP, MongoDB local e build completo do backend. O resumo consolidado está em docs/2TDSPF_2026_Validacao.json.

Parte	Resultado comprovado
Oracle 19.3	12 objetos finais VALID: cinco packages com bodies e dois gatilhos. USER_ERRORS vazio.
Rotinas e auditoria	Exceções esperadas e testes transacionais aprovados, com rollback das alterações de teste.
Dataset real	34 atendimentos exportados do package; 30 realizados em 2026; total R$ 15.635,00.
MongoDB	34 documentos importados e conferidos campo a campo; reimportação manteve 34 documentos.
Java	38 testes executados, zero falhas/erros; 2 testes opcionais PostgreSQL ignorados.
Integração Oracle	Os dois testes OracleIntegracaoTest executaram exportação JSON e faturamento pelo serviço Java real.
JAR	clyvovet-web-4.0.0.jar gerado pelo build completo, com o POM de produção.

Integridade do dataset

SHA-256: 1c5e23089fe23ae726af5c594beb4bad5899a033475fcc098fb17f25430bc04c

Pendências de apresentação: conferir cadastro, atendimento e leitura pela interface Oracle; gravar o vídeo narrado e incluir seu link acessível. Os testes de backend não substituem a demonstração visual exigida.

Continuidade e decisões

O feedback anterior foi “parabéns”, com nota 100. Não existem correções apontadas a aplicar. A evolução conserva as finalidades das duas funções, dos dois procedimentos e da auditoria aprovados; acrescenta organização modular e contratos utilizáveis pelo backend.

Decisão	Justificativa
Oracle como origem	Preserva PL/SQL e packages da entrega anterior.
TB_* e views CV4_*	Java e procedures consultam as mesmas linhas; objetos antigos são preservados.
JSON manual	Conserva o algoritmo aprovado; completa o escape de controles.
OUT CLOB	Permite ao JDBC obter relatórios e JSON sem capturar DBMS_OUTPUT.
Exceções propagadas	A aplicação consegue distinguir erro de um resultado válido.
Transação no chamador	Não há COMMIT nos packages; auditoria acompanha a alteração.
MongoDB documental	Dataset por atendimento com entidades relacionadas aninhadas.

Compatibilidade explicitada

Os nomes físicos evoluíram para os usados no Java, com views CV4_ para leitura pelas rotinas. CACHORRO é apresentado como CAO; CONCLUIDA como REALIZADO. O score Java por regras e o score Oracle por sensores permanecem cálculos distintos.

Empacotamento e contratos

Objeto	Contrato / finalidade
PKG_CLYVO_JSON	FN_MONTA_JSON, FN_VALOR, FN_NUMERO e PRC_IMPRIME_CLOB.
PKG_CLYVO_CLINICO	FN_CALC_RISCO_PET, PRC_CATALOGOS e PRC_REGISTRAR_LEITURA.
PKG_CLYVO_RELATORIOS	PRC_ATENDIMENTOS_JSON e PRC_FATURAMENTO_CLINICA, ambos com OUT CLOB.
PKG_CLYVO_AUDITORIA	Serialização completa do pet e persistência da trilha.
PKG_CLYVOVET	Fachada pública compatível com as chamadas JDBC do Java.
TRG_CV4_AUD_PET	AFTER INSERT/UPDATE/DELETE; delega a auditoria ao package.
TRG_CV4_CONSULTA_CAT	BEFORE INSERT/UPDATE; valida e preenche as FKs de catálogo.

Triggers são objetos independentes no Oracle. O empacotamento solicitado é atendido mantendo o disparador fino e concentrando a lógica reutilizável dentro dos packages.

Exemplo real de chamada do backend

```
{call PKG_CLYVOVET.PRC_ATENDIMENTOS_JSON(?,?)}
{call PKG_CLYVOVET.PRC_FATURAMENTO_CLINICA(?,?)}
{call PKG_CLYVOVET.PRC_REGISTRAR_LEITURA(?,?,?,?,?,?,?)}
{? = call PKG_CLYVOVET.FN_CALC_RISCO_PET(?)}
```

Exceções e transações

Código Oracle	Situação tratada
-20001 / -20002	Identificador de clínica inválido / clínica inexistente.
-20003 / -20004	Ano inválido / conversão do dataset inválida.
-20011 a -20015	Parâmetros JSON, listas divergentes, tipo, valor/tamanho e separador reservado.
-20021 a -20024	Pet inválido, inexistente, sem leituras ou espécie sem suporte ao score.
-20025 a -20028	Catálogo obrigatório, incompatível, inexistente ou ambíguo.
-20029 a -20031	Dispositivo/pet inválido e leitura fora dos limites.
-20051 / -20052	Falha de auditoria / operação inválida.
-20901 a -20903 / -20990	Instalação protegida, compilação, catálogo inconsistente e assertiva de teste.

As falhas de negócio são convertidas por RAISE_APPLICATION_ERROR. Erros inesperados são propagados; não são transformados silenciosamente em sucesso, NULL ou score -1. O backend mostra mensagens de uso sem expor SQL ou credenciais.

Uma clínica existente sem atendimentos retorna []. Um ano sem movimento retorna total zero. Esses dois casos são resultados vazios válidos. Essa evolução em relação à Sprint 3 está documentada.

A auditoria usa valores OLD/NEW escalares e não consulta TB_PET no trigger. O teste transacional valida INSERT, UPDATE, DELETE e desfaz suas alterações ao terminar. Não há transação autônoma na auditoria.

Integração Java e dados compartilhados

O perfil oracle utiliza um único DataSource para JPA e JdbcTemplate. As telas existentes persistem em TB_CLINICA, TB_TUTOR, TB_PET e TB_CONSULTA. As views CV4_ consultam essas mesmas linhas para os relatórios PL/SQL.

O administrador usa /relatorios para exportar o JSON e consultar faturamento. Em /relatorios/oracle registra leituras, chama a função de risco e consulta a auditoria. O agendamento no Oracle oferece os veterinários cadastrados no catálogo.

Arquivo Java	Responsabilidade
RelatorioOracleService.java	CallableStatement com parâmetros vinculados e retorno OUT CLOB.
OracleClinicoService.java	Procedure de inserção de leituras e função de risco; mesmo DataSource JPA.
OracleClinicoController.java	Formulário validado e apresentação dos resultados.
CatalogosOracleAdvice.java	Seleção dos veterinários disponíveis no Oracle.
application-oracle.properties	Migrations exclusivas db/oracle-sprint4 ou instalação manual.

> A versão publicada em PostgreSQL continua disponível, mas não comprova execução de PL/SQL. O vídeo de Database deve usar a aplicação com o perfil Oracle conectado à base FIAP.

A referência usuario_aplicacao da auditoria é opcional: nesta versão, nome_usuario identifica a conta Oracle; não se atribui falsamente esse valor ao login Spring Security.

JSON e modelagem MongoDB

O dataset oficial é produzido pela procedure, lendo as seis entidades relacionais. A versão salva deve ser exportada do Oracle real; dados de exemplo escritos manualmente não substituem o entregável.

```
atendimentos
id_atendimento, data, status, valor
clinica: { id, nome, cidade }
veterinario: { id, nome, crmv, especialidade }
servico: { id, descricao }
pet: { id, nome, especie, raca, sexo, peso_kg }
tutor: { id, nome }
```

Um documento representa um atendimento. Pet, tutor, clínica, veterinário e serviço ficam aninhados porque são consultados junto ao atendimento. Não se acumula um histórico ilimitado dentro do documento do pet.

O MongoDB representa uma projeção dos dados no momento da exportação. Alterar os documentos não atualiza o Oracle. Uma reimportação atualiza os IDs presentes no arquivo; documentos ausentes não são apagados automaticamente.

O importador preserva os campos originais, acrescentando data_bson (UTC) e valor_decimal (Decimal128). O arquivo JSON original não é alterado. O índice único em id_atendimento e o upsert impedem duplicação em reimportações.

O script compara todos os campos de cada documento importado com o arquivo, além da contagem. A coleção usa validação explícita e índices por pet/data e clínica/status/data.

Execução reproduzível

Requisitos: JDK 17+, acesso Oracle 19c+ com quota e privilégios de criação de tabelas, views, sequences, packages e triggers; Docker Desktop com engine Linux para o MongoDB.

```
# Dentro da pasta Database Sprint 4:
.\executar-oracle.ps1 check
.\executar-oracle.ps1 install
.\executar-oracle.ps1 seed
.\executar-oracle.ps1 test
.\executar-oracle.ps1 export
.\executar-mongodb.ps1 iniciar
.\executar-mongodb.ps1 importar
.\executar-mongodb.ps1 consultar
.\executar-oracle.ps1 app
```

O acesso Oracle é solicitado localmente, com senha oculta, ou lido do .env não versionado. O executor Java procura o driver JDBC que já acompanha as dependências do backend.

O instalador verifica a existência dos objetos desta versão e interrompe antes de sobrescrever um projeto existente. Não executa DROP nem modifica os objetos da Sprint 3. A instalação manual e o Flyway são alternativas: não misturar os históricos no mesmo schema.

O Oracle faz commits implícitos em DDL. Uma falha de instalação pode deixar objetos parcialmente criados; não há promessa de rollback completo do instalador. Corrigir a causa e revisar os objetos antes de repetir.

O MongoDB fica vinculado a 127.0.0.1, com volume persistente de demonstração. O comando parar mantém os dados. Não se utiliza --drop nem remoção automática de volumes.

Validação e evidências

Separar o que foi verificado localmente do que depende de serviços externos é essencial. O arquivo docs/VALIDACAO.md registra o estado atual. Não apresentar saídas esperadas como prints reais.

Verificação	Evidência necessária
Packages e triggers	USER_ERRORS vazio e USER_OBJECTS com status VALID.
Mesma base Java/Oracle	Inserir pela interface e consultar o mesmo ID em TB_* e nas views CV4_.
Procedures no backend	Mostrar o resultado da exportação, faturamento e inserção de leitura pela aplicação.
Carga aprovada	34 atendimentos; 30 realizados em 2026; faturamento R$ 15.635,00 antes de novas operações.
Auditoria	OLD/NEW e operação corretos; rollback transacional conferido.
JSON real	Arquivo exportado do package e validado como array JSON.
MongoDB	Importação, comparação dos campos, contagem e reimportação sem duplicatas.
Vídeo	Narração audível e sequência completa sem erros não tratados.

Os testes Java de contrato conferem os parâmetros JDBC e a propagação de falhas. Os testes H2 cobrem a aplicação existente. Nenhum deles substitui compilação e execução reais em Oracle.

Os erros de negócio são provocados apenas nos testes controlados, com código esperado e rollback. No vídeo, usar o fluxo principal validado; não gravar uma instalação com falhas.

Roteiro do vídeo narrado

Trecho sugerido	Ação e fala principal
0:00-0:45	Apresentar ClyvoVet, integrantes e continuidade da Sprint 3.
0:45-1:30	Mostrar modelo lógico/físico e como Java e procedures compartilham TB_*.
1:30-3:00	Cadastrar pet na aplicação; conferir o registro e a auditoria no Oracle.
3:00-4:30	Agendar, iniciar e concluir atendimento; conferir o mesmo ID em TB_CONSULTA e CV4_ATENDIMENTO.
4:30-5:30	Registrar leitura pela tela; explicar a chamada da procedure e consultar o score.
5:30-6:30	Executar faturamento pela aplicação e mostrar a chamada do package no código.
6:30-7:30	Baixar o dataset JSON pela aplicação e mostrar a estrutura aninhada.
7:30-9:00	Importar no MongoDB; conferir documentos, consultas, totais e índices.
9:00-9:30	Reimportar e conferir ausência de duplicação; encerrar com os entregáveis.

Tempos são uma sugestão de gravação; o enunciado fornecido não fixa limite. A narração deve acompanhar as ações: não substituir por música ou texto na tela.

O link final do vídeo só deve ser adicionado depois da gravação e da conferência de acesso pelo professor. O roteiro não equivale ao vídeo entregue.

Conferência dos entregáveis e referências

Item do enunciado	Arquivo / comprovação
Modelos lógico e físico + SQL / 20 pontos	2TDSPF_2026_Modelo_Logico.pdf; 2TDSPF_2026_Modelo_Fisico.pdf; SQL integrado e histórico Sprint 3.
Empacotamento e exceções / 25 pontos	sql/07_packages_gatilhos.sql, incorporado ao SQL integrado.
JSON relacional / 20 pontos	dataset/2TDSPF_2026_Atendimentos.json, após exportação real.
MongoDB / 25 pontos	mongodb/01_criar_estrutura.js, 02_importar_validar.js e 03_consultas.js.
Vídeo / 10 pontos	Gravação narrada e link acessível, após execução validada.

Referências oficiais

Oracle - PL/SQL Packages:
https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/plsql-packages.html

Oracle - CREATE TRIGGER:
https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/CREATE-TRIGGER-statement.html

MongoDB - Modelagem:
https://www.mongodb.com/docs/manual/data-modeling/best-practices/

MongoDB - Instalação Docker:
https://www.mongodb.com/docs/v8.0/tutorial/install-mongodb-community-with-docker/

Código produzido com assistência de IA e sujeito à revisão e compreensão da equipe. Não atribuir contribuições individuais não realizadas.